# ORCA: Observability via Declarative Dataflows

CMU and LANL

2025-05-14

## 1 ORCA: Overview

ORCA (*Observability with Realtime Control and Aggregation*) is designed for continuous, interactive, and realtime visibility into a *bulk-synchronous parallel (BSP)* application. ORCA targets enabling the following workflows:

1. Describe observability intent (a view of application state) via a set of SQL queries (called a `flow`). ORCA orchestrates just the necessary data movement needed to materialize this view, using SQL operator placement.
2. Dynamically update this intent, by installing a new `flow`. All subsequent telemetry is aggregated using the new flow.

These two capabilities enable an entire spectrum of observability needs: from common-case monitoring (minimal collection of summary statistics) to root-cause localization if and when a problem is suspected.

## 2 Motivation: Lessons from Optimizing Parallel Codes

We spent 2+ years understanding parallel code performance at upto 4,096 cores on an in-house cluster and optimizing it. ORCA emerged from this experience. Key lessons:

1. Global collectives impose extreme performance constraints: a single straggler at a global barrier can cause underutilization of the entire system (see Fig. 1).
2. Sampling and profiling, popular aggregation approaches in existing tooling, obscure problem structure (see Figs. 1 and 2).
3. Fine-grained telemetry is currently only available via tracing. Tracing formats are not amenable to ad-hoc analytics, and their post-mortem nature delays problem localization.

The following needs, thus emerged, for an ideal observability system:

1. Solving performance problems requires synthesizing meaningful views from raw telemetry. eBPF enables this at a single-node, but not at cluster-scale.
2. View definition interface should be programmable and dynamically updatable. Even with tracing, it is not possible to collect *all* telemetry, nor is it necessary.
3. Recurrent performance problems require a top-down analysis, to be able to go from *"is my app slow?"* to *"what in the stack is causing this?"*.

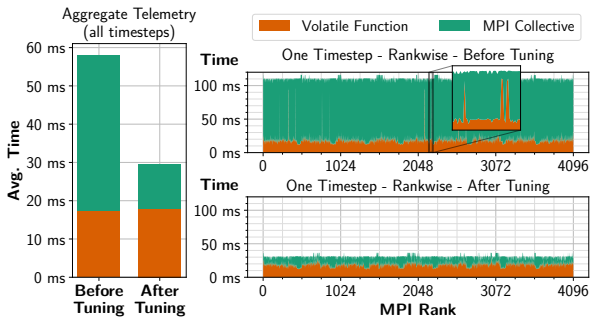


Figure 1: Simplified view of telemetry from a real BSP app with a volatile function followed by a global collective. Tail latency in volatile function gets misattributed as elevated collective time in aggregate telemetry (left), but not in fine-grained telemetry (right). Localization revealed an issue in the network stack, mitigating which eliminated the spikes, and therefore the collective time.

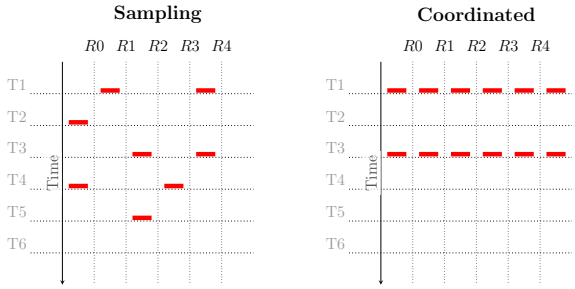


Figure 2: A visualization of the two dimensions of BSP telemetry: timesteps (epochs), and ranks (workers). Sampling collects an arbitrary subset of telemetry. Coordinated collection is necessary to understand problem structure.

We found BSP applications to be highly amenable to SQL-based aggregation (`GROUPBY` timestep or epoch, compute summary statistics). We decided to model telemetry as relational data, and leverage modern OLAP developments: Arrow, Parquet, and Datafusion to build an observability engine using in-situ in-memory aggregation for dynamic dataflows.

# 3 ORCA: Aggregation and Control

ORCA leverages an overlay network to mediate between an active BSP application and an asynchronous controller. The overlay augments the controller with a tier of dedicated aggregator nodes (collectively comprising the *asynchronous/monitoring domain*), which monitors a BSP application (Fig. 3). Thus a tree topology is created, with leaf nodes being MPI ranks, which are sharded and assigned to aggregators. An intermediate tier of lead MPI ranks can be used for lightweight pre-aggregation to manage fanout at extreme scales.

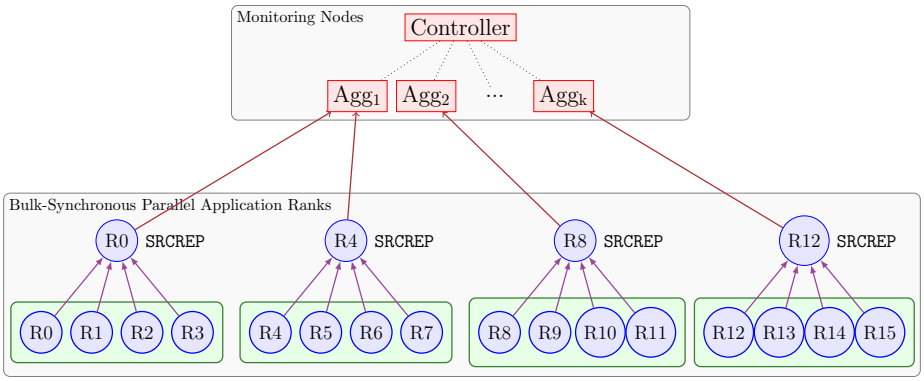


Figure 3: An overlay example showing the aggregation flow between MPI ranks and the monitoring tier, going to controller via MPI reps and aggregators.

The overlay communicates via Mercury, an RPC library for high-performance fabrics. ORCA is amenable to all its communication being relegated to an asynchronous low-QoS tier, so as to not interfere with application performance. Observability capabilities are built on two key functions:

1. Bottom-up aggregation in a manner defined by the active `flow`, with operator pushdown to minimize data movement, using the overlay as a reduction tree.
2. Control semantics, enabled by an asynchronous, timestep-consistent two-phase commit protocol (`TS2PC`). `TS2PC` enables replicated state machine semantics on the MPI tier, using the overlay as a broadcast tree. Brief `TS2PC` description:

   - *Correctness guarantee*: control commands commit on all ranks or none.
   - *Performance guarantee*: Protocol execution is asynchronous and non-blocking. If the controller thinks that the application is at timestep $T$, a control message is issued to be applied at timestep $T + \Delta$. $\Delta$ is a function of overlay latency and is learned automatically.
   - *Usage*: `TS2PC` is used to issue commands such as `activate/deactivate probe`, `pause/resume` application, `update flow` etc.

## 4 Design Overview

1. *Data collectors* represent emittable streams with different schemas. We currently have data collectors for MPI collectives and Kokkos, and more can be added for `NCCL`, `Pytorch`, eBPF, Infiniband etc. Each data collector may emit different probes, each of which can be individually toggled at runtime. If probes are disabled or stream is not needed, the collection call becomes a no-op.
2. Data collectors are flushed at the end of each timestep to generate Arrow data, which is collected and aggregated via the tree. Aggregation also sequences collected data by timestep, to ensure that a prior timestep is completed before the next timestep is processed.
3. Each layer applies assigned partial aggregations, and pushes the results up the tree. Finally, data is collected to different *sinks*: these could be on-disk Parquet files, timeseries databases serving Grafana etc.

ORCA is written in C++, with query planning/execution in Rust using Apache Datafusion.

## 5 Potential Extensions

We briefly describe some potential extensions to ORCA. A demonstration of the first two is in our roadmap.

1. *Online operator-in-the-loop tuning*: the `TS2PC` mechanism can be used to update various application parameters (thresholds, rates etc.) at runtime (must be exposed via ORCA), and observe their impact in realtime. ORCA can also form the infrastructure hosting autotuning algorithms, providing them control and feedback.

2. *Observing application state*: in addition to performance telemetry, ORCA can also aggregate internal application state (such as statistics on particle energies or model weights and gradients). Approximation techniques such as sketches can be used to define lossy aggregations, to reduce data volume with minimal impact on fidelity.

3. *Leveraging accelerators*: ORCA can be extended to use accelerators as intermediate nodes. Their capabilities can be exposed as special nodes in the dataflow, and the planner can be extended to leverage them. Datafusion's architecture enables continuous evolution of query planning and execution.